

【全栈工程师】Test Case - 2026.Q2

你的测评 project 是为金融、法律、会计等职业资格考试学习者设计并实现一个智能备考平台。系统面向 CFA、FRM、CPA、法律职业资格等高知识密度考试，帮助学习者通过结构化知识体系、互动式练习、即时反馈、知识点掌握度追踪、错题复习和个性化学习路径，提高长期备考效率。

重要：我们重视候选人对系统架构扩展性的理解。请展现你设计可扩展、可维护系统的能力。

- **时间限制：**7天
- **AI / API 使用：**候选人可自行使用 AI 编程工具与 OpenAI API。相关费用可申请补贴，提交时请附账单截图或调用记录。
- **数据库：**后端数据库一律使用 Supabase (PostgreSQL)。
- **环境变量：**GitHub 仓库中只提交 .env.example。真实 .env 请通过 Email、加密压缩包或其他安全方式单独提供。
- **二次面试 - 现场 Demo：**清楚说明数据流、查看数据库结构、说明缓存项目、后台任务设计、学习数据建模和学习智能化进阶逻辑。

挑战结构

第一部分：系统架构设计（必须完成）

针对能够管理考试知识体系、追踪学习表现、识别薄弱知识点，并提供个性化学习路径的平台，设计一个完整的全栈系统架构，包含以下文件：

1. API 设计文件
 - RESTful API endpoints 设计
 - Request / Response 格式定义
 - 认证与授权机制
 - 用户角色与权限设计（学习者、教研/内容编辑、管理员）
 - Rate limiting 策略
 - 错误处理规范
2. 数据库设计
 - 数据表 schema（使用 PostgreSQL）
 - 考试、科目、章节、知识点、题目之间的关系设计
 - 学习记录、答题记录、错题本、掌握度模型或其他学习智能化数据结构
 - 索引策略
 - 数据一致性保证
3. 系统架构图

- 前后端组件关系图
- 学习数据流向图
- 学习智能化流程图（如每日路径、任务选择或交互式学习单元）
- 部署架构（考虑水平扩展）

4. 缓存与任务设计

- Redis 缓存策略（哪些数据要缓存、TTL 设置）
- 后台任务设计（使用 Celery、BullMQ、Sidekiq 或类似工具）
- 学习报告生成、路径生成、任务选择等批处理架构
- 失败重试与幂等性设计

5. 监控与运维

- Log 架构设计
- 关键指标定义（API 延迟、任务成功率、推荐命中率、答题数据写入成功率）
- 错误追踪与告警机制
- 成本监控设计（AI/API 调用、后台任务、数据库读写）

第二部分：核心功能实现（必须完成）

请完成以下 2 个基础功能。

必做功能 1：考试知识体系与题库管理系统

需求：

- 支持基础角色设计：管理员、教研/内容编辑、学习者
- 管理员可管理考试结构与用户权限；教研/内容编辑可维护题目与解析；学习者只能查看已发布内容并参与练习
- 支持考试、科目、章节、知识点、题目的层级管理
- 每道题需包含题型、难度、答案、解析，并关联一个或多个知识点
- 支持题目批量导入，以及按考试、科目、章节、知识点查询题目

技术要求：

- 使用 Supabase (PostgreSQL) 建模
- 提供题库管理 API 与最小可用前端管理界面
- 设计合理索引，支持高频题库查询

必做功能 2：练习、判题与学习记录系统

需求：

- 学习者可以选择考试、科目、章节或知识点开始练习
- 支持单选题、多选题、判断题中的至少 2 种题型
- 提交答案后返回判题结果、正确答案、解析、关联知识点和答题耗时
- 自动记录答题行为，错题进入错题本

- 支持查看基础进度，例如已完成题数、正确率、最近学习日期、错题数量

技术要求：

- 设计答题 session 与答题记录数据结构
 - 前端需要有完整练习流程
 - 后端需要保证重复提交、刷新页面、网络重试时的数据一致性
 - 提供答题结果、学习记录、错题查询 API
 - 需要为答题记录写入、错题归档逻辑编写单元测试
-

第三部分：学习智能化进阶（三选二）

从以下 3 个进阶方向中选择 2 个进行实现。

进阶方向 1：每日学习路径与习惯留存系统

目标：把长期备考拆成短小、明确、每天可完成的学习路径。

需求：

- 学习者每天进入后看到一条明确的今日学习路径
- 今日路径包含 2-4 个短任务，例如新题、错题、复习、概念回看
- 系统根据最近表现调整今日路径，而不是固定任务清单
- 每完成一步都有即时反馈，例如进度推进、XP、完成状态或奖励
- 支持连续学习记录、中断保护或恢复任务
- 展示最近 7 天学习节奏

技术要求：

- 设计学习路径、任务步骤、学习事件、连续学习记录的数据结构
- 路径生成需考虑考试日期、最近表现、错题和未完成任务
- 任务完成、XP/奖励、连续学习计算需要幂等
- 前端需展示今日路径、逐步完成反馈和连续学习状态

进阶方向 2：诊断测评、知识图谱与自动任务选择引擎

目标：用诊断和知识点图谱判断学习者当前水平，由系统选择下一步最值得学习的任务。

需求：

- 支持入门诊断测评，生成初始强弱项
- 建立知识点图谱，包含知识点、前置关系和题目映射
- 根据诊断和答题记录维护学习者知识画像
- 自动选择下一步任务：新学、补前置、错题复习或间隔复习
- 每条任务必须说明选择依据，例如前置未掌握、近期答错、接近遗忘周期

技术要求：

- 设计诊断测评、知识图谱、知识画像、任务选择记录的数据结构

- 任务选择逻辑需要可解释，并有单元测试
- 说明如何在新学、补前置、复习、推进章节之间取舍
- 前端需展示知识点状态、推荐任务和推荐原因

进阶方向 3：交互式概念探索与即时反馈系统

目标：把复杂概念或题目做成可交互学习单元，让学习者通过操作、判断和反馈建立理解。

需求：

- 至少实现 1 个交互式学习单元，围绕一个复杂知识点或题目展开
- 学习单元需包含 3-6 个交互步骤，例如拖拽公式组件、调整图表/计算参数、排序推理链、匹配概念卡片、标记关键条件、选择或填空
- 每一步根据用户操作即时反馈，而不是最后统一给解析
- 答错时提供当前步骤提示，并解释可能误区
- 最后生成概念卡片或总结

技术要求：

- 可以使用 OpenAI API 分析学习数据与题目内容
- 结构化保存交互步骤、用户操作、反馈、提示和概念卡片
- 如使用 AI，输出需包含步骤、交互类型、反馈、误区解释等字段
- 如使用 AI，输出需要结构化保存，并保留生成依据和版本
- 如使用 AI，需要处理调用失败、超时、返回格式异常等情况
- 前端必须实现真实交互流程，不能只展示分段文字解析

提供的资源

- 鼓励使用 AI 编程工具辅助开发，但架构设计和技术取舍需要体现候选人自己的思考。
- 如使用 OpenAI API，请候选人自行准备 API Key，并通过 .env 配置。
- AI / API 相关费用可申请补贴，提交时请附账单截图或调用记录。
- Supabase 使用要求
 - 可使用免费方案。
 - 请提供可重现的数据表创建脚本，或提供 Supabase 项目的读取/查看权限。
- 题库数据格式示例
 - 候选人需自行准备 30-100 道金融、法律或会计类示例题。
 - 题目必须具备明确答案、解析和知识点标签。
 - 不接受完全无意义的 mock data。

重要提醒

所有 Demo 数据必须具备真实业务语义：

- 题目、解析、知识点标签需要符合金融、法律或会计类考试的真实学习场景

- 不接受只有 test question 1 、 sample answer 等无意义数据
- Demo 时需要展示所选功能对应的完整学习流程，例如选题、答题、反馈、记录、分析、今日路径、任务选择或交互式学习单元

交付要求

1. 架构设计文件
 - ARCHITECTURE.md 、 API_DESIGN.md 、 DATABASE_DESIGN.md
 - LEARNING_INTELLIGENCE_DESIGN.md
 - 架构图 (draw.io、Mermaid 或类似工具)
2. 代码实现
 - 第二部分 2 个必做基础功能
 - 第三部分选择的 2 个进阶方向
 - 单元测试 (至少 70% coverage)
 - Docker Compose 与 .env.example
3. 前端体验
 - 覆盖所选功能的核心用户流程
 - 展示答题反馈、学习进度，以及所选进阶方向的关键体验
4. 文件说明
 - README.md 、 DESIGN_DECISIONS.md
 - API 文件 (Swagger 或 Postman collection)
5. Demo 视频 (5-10 分钟)
 - 讲清架构、功能流程、数据流和扩展性

Demo 场景选择

建议选择一个具体考试方向，例如 CFA、FRM、CPA 或法律职业资格考试。

评分标准

项目	权重	评估重点
系统架构设计	40%	可扩展性、可维护性、性能考量、学习数据建模
代码质量	30%	遵循 SOLID、DRY、代码组织、测试覆盖
问题解决能力	20%	技术决策、错误处理、边界案例、进阶学习能力的可解释性
文件完整性	10%	清晰度、完整性、可执行性

重点看基础学习闭环是否稳定可用，以及进阶方向是否真正服务于每日路径、任务选择或交互式理解。

加分项目

- 完整实现第三部分全部 3 个进阶方向
- 知识图谱可视化或学习策略效果评估
- 性能优化、安全设计、CI/CD、监控设计
- AI/API 成本控制与缓存策略

补充规范 (Submission Add-ons)

- 使用 Supabase
 - 后端数据库必须使用 Supabase (PostgreSQL)
 - 提供 Supabase 只读权限，或提供可重现 SQL schema
 - 附 ERD 图或 SQL schema dump
- 提交 GitHub 的同时，单独提供可用的 .env 文件
 - GitHub 只提交 .env.example
 - 真实 .env 请通过加密压缩包、Email 或其他安全方式单独提供
- 二面现场 Demo 要求
 - 说明数据流、数据库设计、缓存/后台任务、关键算法或规则
 - 准备一键启动指令和最小测试数据

注意事项

1. 架构优先：请先完成架构设计再开始实现
2. 代码质量：使用 linter 和 formatter
3. 关键逻辑必须有测试
4. 测试数据必须符合真实考试学习场景
5. 推荐结果、AI 输出和进阶规则需要可解释

提示

- 不要做成简单题库 CRUD，重点体现学习路径、任务选择和交互式理解
- 考虑考试日期、学习时间、知识点依赖、遗忘和题目难度
- 保持 KISS、DRY、YAGNI，避免过度设计
- 如使用 AI，输出需结构化、可追踪，并有失败兜底

常见问题

Q: 必须实现所有功能选项吗？

A: 不用。第二部分 2 个基础功能必须完整实现，第三部分从 3 个学习智能化进阶方向中选择 2 个完整实现即可，但架构设计文件需要覆盖完整平台。

Q: 数据库一定要用 PostgreSQL 吗？

A: 是的，后端数据库需要使用 Supabase (PostgreSQL)，可以搭配 Redis 作为缓存。

Q: 可以使用 AI 工具辅助开发吗？

A: 可以。我们评估的是最终结果、架构设计、代码质量和技术决策。

祝你挑战顺利！我们期待看到你对智能学习系统的架构设计能力、工程实现能力和产品理解。